Sleep Apnea Machine Learning Project Specification

Version 1.0 | 10 September 2026 | Phase 1 research comparison

Purpose and research question

Build a reproducible experiment that measures how much overnight sleep apnea severity classification performance changes when a model uses a reduced set of physiological signals instead of the full eligible signal set. Compare both versions against the same dataset reference labels and on the same held-out participants.

The study measures the observed performance gap. It does not assume equivalence, impose an acceptable loss threshold, or establish that either model replaces clinical assessment. A small gap must be interpreted alongside both models' absolute performance.

Scope and decision status

Agreed requirements include four severity categories, one prediction per night, awake periods retained, window-based processing, paired full and reduced inputs, and overnight analysis on a laptop. MESA Sleep is the initial dataset candidate. Data access and usable signals remain unverified.

Proposed defaults in this specification are implementation recommendations, not additional decisions already confirmed during the interview. Dataset-dependent choices must be recorded and fixed before final testing.

Shared terminology

Term	Meaning
Reference label	Dataset-derived overnight category: none, mild, moderate, or severe. The exact scoring definition remains pending.
Full inputs	All agreed eligible physiological signals for this experiment, excluding labels and human-scored outcome information.
Reduced inputs	All available target signals together: heart rate, SpO_2, six-axis IMU, and microphone audio. A subset of the full inputs.
Window	A short processing segment. Windows are combined to produce a night-level prediction; they are not independent diagnosed examples.
Model pipeline	Preprocessing, feature construction or representation learning, training, and night-level prediction.
Checkpoint	Saved trained model plus the preprocessing and metadata required to reproduce inference.
Performance gap	Full-input score minus reduced-input score within the same model family. Positive values favor full inputs.
Variance	Variability across repeated training runs; distinct from the performance gap and its uncertainty.

Data and prediction requirements

Dataset intake

Use one selected dataset for the initial comparison. MESA is the candidate, not a confirmed inventory. Before training, inspect access conditions, dataset release, participant and recording identifiers, available channels, sampling rates, units, timestamps, missingness, and label documentation. Record the included cohort and reasons for exclusion.

Signal or annotation	Planned treatment
SpO_2 and heart rate	Reduced and full inputs when available. The initial desired rate is 1 Hz; actual rates and measurement sources must be verified.
Six-axis IMU	Reduced and full inputs if present: acceleration x/y/z and angular velocity x/y/z. Body position and activity counts are not equivalent substitutes.
Microphone audio	Reduced and full inputs if synchronized audio is present. A derived snore indicator is not assumed to be raw audio.
ECG, airflow, respiratory effort	Candidate full-only inputs. Include thoracic and abdominal effort if available. ECG-derived heart rate may be retained in the reduced set, but raw ECG must not enter it.
EEG, EOG, EMG, body position	Candidate full-only inputs, subject to availability and agreed eligibility.
Sleep stages and breathing annotations	Excluded from prediction inputs. May support reference-label construction and later sleep-filtering experiments.

Freeze explicit full and reduced channel lists for each experiment. If audio or IMU is unavailable, adapt the reduced set and name the tested combination accurately. The result cannot establish the contribution of absent signals. Do not change the cohort between the two input versions.

Reference labels

Choose one documented severity definition and fixed class order: none, mild, moderate, severe. Record the source variable, event-scoring rule, category boundaries, and treatment of missing or ambiguous labels. If an apnea–hypopnea index is used, record its event and sleep-time definitions. Do not silently combine incompatible label definitions.

Clarify whether the selected outcome represents overall sleep-disordered breathing or a specific apnea subtype. Use that definition consistently in the report. Human-scored labels and their direct derivatives must not appear among model predictors.

Windows and quality handling

Retain awake periods. Synchronize channels and process the night in windows, with duration and overlap selected using development data. Preserve suitable channel-specific rates; do not assume ECG or audio can be treated as 1 Hz signals. Record resampling, filtering, missing-value, artifact, and minimum-valid-duration rules.

Quality exclusions may remove unusable segments but must not silently implement sleep filtering. Never assign the overnight severity label to every window as though it were a window-level diagnosis. The supervised target and final evaluation remain at night level.

Model and training protocol

Model lineup

Pipeline	Input representation	Status
Gradient-boosted trees	Calculated window features summarized into one feature vector per night.	Selected
Deep learning	Signal window encoder and night-level aggregation, trained against overnight labels. Exact architecture pending.	Selected
Multinomial logistic regression	Same calculated feature definitions as the other conventional models; fitted scaling and regularization.	Proposed addition
RBF support vector machine	Same calculated features; fitted scaling and kernel settings.	Proposed addition

With all four pipelines, the main experiment has eight selected checkpoint bundles: four model families multiplied by full and reduced inputs. Tuning trials and repeated runs create additional artifacts; eight is the count of primary model/input combinations, not a cap on saved files.

Paired experimental design

- Create participant-disjoint training, validation, and test partitions. All nights and windows from a participant stay in the same partition. Preserve identical partitions across all pipelines and both input versions.
- Use the same eligible recordings and common quality policy for paired comparisons. Fit scaling, imputation, feature selection, and other learned preprocessing only on training data within each development split.
- Train each full and reduced version separately. Proposed default: tune each with the same declared trial budget, development splits, selection metric, and comparable early-stopping rules. Record search spaces and actual compute use.
- Select settings and checkpoints using validation macro-F1. Do not use test results to choose models, windows, thresholds, preprocessing, or class weighting.
- Freeze the analysis configuration before final testing. Evaluate every selected model/input combination on the held-out test set and retain all results, including weak results.

Representation and reproducibility

Conventional pipelines use a shared feature specification restricted to each input list. Candidate features may describe levels, variability, temporal changes, and signal-specific patterns. Features requiring excluded channels or expert annotations are prohibited in the reduced pipeline.

The neural pipeline must combine window representations into a night-level output and optimize the night-level objective. Padding, masking, window sampling, and maximum recording length must be documented. Final inference must follow the same saved aggregation policy.

Record dataset and configuration versions, participant split identifiers, random seeds, software versions, hardware, runtime, and model settings. A laptop with a GPU is available; GPU model and memory remain unknown. Choose batch sizes and architecture after inspecting these constraints.

Repeated runs

Proposed default: repeat stochastic training under a predeclared set of seeds, using matched seeds for full and reduced runs. Keep the test cohort fixed and report paired run differences. Determine the repeat count from the available compute before final testing; deterministic pipelines do not need artificial repeats.

Evaluation and reporting

Scoring contract

Generate one predicted severity category per eligible test night. Fix the class order across every output. Macro-F1 is the unweighted mean of the four class-specific F1 scores. Compute each performance gap by subtracting the reduced score from the full score for the same model family and evaluation cohort.

Output	Required content
Overall scores	Macro-F1 as primary; accuracy, macro precision, and macro recall as supporting scores.
Separate category scores	Precision, recall, F1, and reference-label count for each of the four severity categories.
Confusion matrices	Raw counts and row-normalized proportions using the same class order. Highlight the direction and extent of misclassification.
Paired performance gaps	Absolute full-minus-reduced differences for the reported comparable scores, including the primary macro-F1 gap.
Prediction records	Participant ID, recording ID, reference category, predicted category, model, input set, run ID, and class scores when available.
Uncertainty and variability	Proposed: paired participant bootstrap intervals for gaps; mean and standard deviation across repeated training runs when performed.

Report gaps on the 0–1 score scale and, optionally, as percentage-point differences. For example, 0.82 minus 0.79 is a gap of 0.03, or 3 percentage points; it is not a 3 percent relative decrease. This example is illustrative, not an expected result.

Uncertainty protocol

Proposed default: a 95 percent paired bootstrap interval for the primary gap. Resample test participants with replacement and include all their retained nights. Use exactly the same resampled participants for full and reduced predictions, and recompute macro-F1 for each resample. Never bootstrap windows as independent people.

Sampling uncertainty and training variability must be reported separately. Fix the number of bootstrap resamples, treatment of class-absent resamples, and aggregation across training seeds before testing. An interval containing zero does not prove equivalence.

Edge cases and interpretation

Require all four reference categories in validation and test data when feasible. If a category is absent, flag the evaluation as incomplete rather than implying a reliable four-category result. Proposed convention: undefined precision or F1 is recorded as zero for aggregation, accompanied by an explicit flag and class counts.

If several nights per participant exist, preserve participant-level splits and uncertainty resampling. Predeclare whether the primary cohort uses one night per participant or all eligible nights; do not let participants with more recordings receive unexamined extra weight.

The report must show absolute performance and gaps together, dataset exclusions and class balance, failure cases, and uncertainty. Discuss results as specific to the tested cohort, label definition, signal sources, and pipelines. An ECG-derived heart-rate result does not by itself validate wristband-derived heart rate. Full-input model predictions are comparators, not reference labels.

Software deliverables and completion criteria

Proposed file interfaces

The following names define the intended software structure. Implementations may share modules; separate copies of preprocessing code are not required for each checkpoint.

File or module	Responsibility
configs/experiment.yaml	Dataset and label definitions, channels, splits, windows, model settings, tuning budgets, seeds, metrics, and output locations.
prepare_data.py	Validate data and labels; create channel inventory, cohort manifest, quality report, and participant split manifest.
run_experiments.py	CLI coordinator for configured model × input combinations. Support resumable runs, explicit statuses, and saved failures.
train_model.py and models/	Individual training entry point and model-specific implementations using shared data preparation.
evaluate.py	Load selected checkpoints, run held-out inference, and save predictions and metrics without refitting preprocessing.
generate_report.py	Generate tables, confusion matrices, paired comparisons, and a readable report from saved results.
checkpoints/ and results/	Checkpoint bundles; metrics.csv, predictions.csv, run_metadata.json, uncertainty results, and final report.

Each checkpoint bundle must include model parameters, learned preprocessing, ordered features or channels, units, sampling/window rules, class mapping, configuration, split identifier, and training metadata. Reloading a checkpoint must reproduce its saved test predictions within a declared numeric tolerance.

Metrics records must identify dataset version, cohort/split, pipeline, input set, seed/run, metric, class if applicable, score, and sample count. Metadata must list excluded modalities and completed, failed, or skipped runs. Class scores must be identified as probabilities or decision scores, as appropriate.

Acceptance and remaining decisions

Acceptance criteria

- The data manifest documents label rules and exact full/reduced channels; automated checks confirm no participant overlap or forbidden predictors.
- The coordinator completes all configured pairs or records actionable failures. Every successful selected checkpoint can be loaded independently for inference.
- Paired models have matching test recording IDs. The evaluator produces the agreed overall and per-category metrics, confusion matrices, and reproducible gaps.
- The report distinguishes measured results from pending capabilities and includes the proposed uncertainty analysis if adopted. No target F1 or maximum acceptable gap is required for project completion.

Decisions to resolve before training

Confirm dataset access and release, actual channels, severity definition and apnea subtype, cohort size and repeated-night handling, split proportions, quality rules, window settings, neural architecture, additional model adoption, GPU capacity, and tuning/repetition budgets. Record these in the experiment configuration before final testing.

Deferred work

Wristband transfer and validation, sleep-only filtering, multi-night aggregation, live sleep/wake detection, Home Assistant, and IR control are outside this phase. No automation architecture is specified here.